

1. Overview

Scenario.

You are building a backend for an application used by a Romanian insurance company that manages **building insurance policies**.

There are two main user groups:

1. **Brokers**
 - Manage clients.
 - Manage buildings (objects of insurance).
 - Create and manage policies.
 2. **Administrators**
 - Maintain metadata (currencies, fees/coefficients, risk factors).
 - Manage brokers (activate/deactivate).
 - View reports (e.g., number and total value of policies per country/county/city, per broker, per period).
-

2. Learning goals

By implementing this exercise, the learner should practice:

1. Designing clear **domain models** and relationships.
 2. Exposing **REST API endpoints** for different user roles.
 3. Implement according to API first - design approach
 4. Handling **basic business rules** around policies and premiums.
 5. Writing code that is **easy to extend**: separation between domain, data access, and Web API.
 6. Implementing **unit tests** for core logic and **integration tests** for API endpoints and persistence. At least 80% code coverage is required (this percentage can grow depending on the technical stack used)
 7. Thinking about **code quality**, **readability**, and **maintainability** from the start.
-

3. Domain overview & core concepts

The system manages:

- **Clients** – people or companies that own buildings and buy policies.

- **Buildings** – the insured objects.
 - **Policies** – contracts covering buildings for a period, with a premium and currency.
 - **Geography** – country, county, city.
 - **Brokers** – intermediaries that sell policies.
 - **Administrators** – internal staff who manage configuration and reporting.
 - **Metadata** – currencies, fees/percentages, risk coefficients, etc.
-

4. Core models & relationships

Below is a **business-focused description** of models and how they relate. (Implementation details like classes and tables can be derived by developers.)

4.1 Policies

- **Policy**
 - Describes the **insurance contract**.
 - Fields:
 - Unique policy number (e.g., generated string).
 - Client (the policy holder).
 - Building (object of insurance).
 - Broker (who sold the policy).
 - Policy status (Draft, Active, Expired, Cancelled).
 - Start date, End date.
 - Base premium amount.
 - Currency (reference to Currency metadata).
 - Calculated final premium (after fees and risk adjustments).
 - Creation date, last update date.
 - A **Policy** belongs to exactly one **Client**, one **Building**, and one **Broker**.
 - A **Building** can be linked to multiple **Policies** over time, but at a given moment it might be covered by 0, 1 or multiple policies (depending on rules you choose).

Relations:

- Policy (many) ↔ (1) Client.
 - Policy (many) ↔ (1) Building.
 - Policy (many) ↔ (1) Broker.
 - Policy (many) ↔ (1) Currency.
 - Policy may reference fee configurations (see metadata below) used at calculation time.
 - Building (1) → (many) Policy (each policy covers exactly one).
-

4.2 Brokers and administrators

- **Broker**
 - Represents a **person or organization** selling policies.
 - Fields:
 - Unique identifier.
 - Broker code.
 - Name.
 - Email, phone.
 - Status (Active / Inactive).
 - Commission percentage (optional, if required).
 - A **Broker** can manage **Clients** and create many **Policies**.
 - **Administrator**
 - Represents an internal staff member.
 - For a beginner exercise, can be simplified:
 - Unique identifier.
 - Name.
 - Email.
 - Role (e.g., Admin, Manager).
 - Used conceptually for admin endpoints (no full-blown auth needed in the brief, but can be implemented if case it does not impact to much the scope of this sprint).
-

4.3 Metadata

- **Currency**
 - Fields:
 - Code (e.g., RON, EUR).
 - Name.
 - ExchangeRateToBase (decimal)
 - IsActive flag.
 - Used on **Policies** to express premium amounts.
- **FeeConfiguration**
 - Represents adjustable **fees/percentages**.
 - Fields:
 - Name (e.g., “Standard broker fee”, “Earthquake risk adjustment”).
 - Type (BrokerCommission, RiskAdjustment, AdminFee, etc.).
 - Percentage (decimal).
 - EffectiveFrom / EffectiveTo (validity period).
 - IsActive flag.

- Admins can create/update these; policy calculations can use the currently active configurations.

➤ **RiskFactorConfiguration** (optional/simple)

- Specifies extra percentage adjustments by **geography or building features**.
- Example fields:
 - Level: Country / County / City / BuildingType.
 - ReferenceId (e.g., specific County or City).
 - AdjustmentPercentage (positive or negative).
 - IsActive flag.

Policies can use **FeeConfiguration** and **RiskFactorConfiguration** to derive a final premium. For a beginner exercise, you may keep the calculation simple (e.g., base premium $\times$ (1 + sum of percentages)).

5. Functional requirements – Broker-facing endpoints

These are written from a **business analyst** perspective; actual routes, HTTP verbs, etc., are at the developers' discretion.

5.1 Policy management

1. Brokers must be able to **create a draft policy**:
 - Select client.
 - Select building owned by the client.
 - Select currency.
 - Provide base premium amount.
 - Provide policy period (start and end dates).
 - Automatically associate the logged-in broker (for now can be passed as input).
2. When a draft policy is created, the system should **calculate a preliminary final premium**:
 - Using active fee configurations.
 - Using active risk factor adjustments (e.g., by city or building type).
3. Brokers must be able to **activate** a policy (transition from Draft to Active), if:
 - Start date is not in the past (or allowed per business rule).
 - All mandatory fields are present.
4. Brokers must be able to **cancel** a policy, with a reason:
 - Only Active policies can be cancelled.
 - Effective cancellation date should be recorded.
5. Brokers must be able to **list policies** by:
 - Client.
 - Broker.
 - Status (Draft, Active, Expired, Cancelled).

- Date range (start/end date in range).
 - 6. Brokers must be able to **view policy details**, including:
 - Client and building details.
 - Premium amounts (base, final).
 - Currency.
 - Geography (through building's location).
-

6. Functional requirements – Admin-facing endpoints

6.1 Metadata management (Currencies, Fees, Risk factors)

1. Administrators must be able to **manage currencies**:
 - List all currencies.
 - Add a new currency (code, name, exchangeRateToBase, isActive).
 - Activate/deactivate currency (cannot deactivate if used in active policies, or at least prevent future use).
 2. Administrators must be able to **manage fee configurations**:
 - List fee configurations.
 - Create new fee configuration with:
 - Name.
 - Type.
 - Percentage.
 - EffectiveFrom / EffectiveTo.
 - IsActive.
 - Update existing configuration (e.g., percentage).
 - Deactivate configurations no longer in use.
 3. Administrators must be able to **manage risk factor configurations**:
 - Define risk adjustments by county, city, or building type.
 - Set adjustment percentage.
 - Activate/deactivate adjustments.
-

6.2 Broker management

1. Administrators must be able to **create a new broker**:
 - Broker code.
 - Name.
 - Contact info.
 - Initial status (Active / Inactive).
2. Administrators must be able to **update broker details**.
3. Administrators must be able to **activate/deactivate brokers**:
 - Deactivated brokers must not be allowed to create new policies.
 - Existing policies remain visible.

7. Non-functional & code-quality requirements

The code should aim for:

1. **Separation of concerns**
 - Keep business logic out of controllers.
 - Use services or application layer classes to handle policy calculations.
 2. **Maintainability**
 - Clear naming.
 - Small methods with single responsibility.
 - Avoid duplication (e.g., one coherent policy calculation service).
 3. **Scalability (at a basic level)**
 - Avoid loading too much data into memory unnecessarily.
 - Use paging for listing endpoints (clients, policies, etc.), even if simple.
 4. **Extensibility**
 - New metadata types or risk adjustment rules should require **minimal changes**.
 - Policy calculation logic should be centralised so new fees/rules can be added.
 - Use design patterns (e.g. Strategy) to structure your code to adapt to new features.
 5. **Validation**
 - Validate incoming data:
 - Required fields (e.g., Broker code and name, currency code, etc.).
 - Reasonable ranges (e.g., positive premiums, insured value > 0).
 - Return clear error messages.
 6. **Error handling and logging**
 - Centralised error handling (e.g., a middleware or equivalent).
 - Log key events: policy creation, activation, cancellation, failures.
 7. **Security (basic level)**
 - Assume that some form of authentication will be used.
 - Endpoints for brokers and admins should be **clearly separated**, ready to apply authorization rules.
-

8. API specification (high-level)

Examples (developers can choose exact routes):

8.1 Broker – Policies

- GET
/api/brokers/policies?clientId=&brokerId=&status=&startDate=&endDate=
- GET /api/brokers/policies/{policyId}

- `POST /api/brokers/policies // create draft`
- `POST /api/brokers/policies/{policyId}/activate`
- `POST /api/brokers/policies/{policyId}/cancel`

Responses should include clear status codes:

- 200 OK (success).
 - 201 Created (for creations).
 - 400 Bad Request (validation errors).
 - 404 Not Found.
 - 500 for unexpected server errors.
-

8.2 Admin – Metadata

- `GET /api/admin/currencies`
- `POST /api/admin/currencies`
- `PUT /api/admin/currencies/{id}`
- `GET /api/admin/fees`
- `POST /api/admin/fees`
- `PUT /api/admin/fees/{id}`
- `GET /api/admin/risk-factors`
- `POST /api/admin/risk-factors`
- `PUT /api/admin/risk-factors/{id}`

8.3 Admin – Brokers

- `GET /api/admin/brokers`
- `GET /api/admin/brokers/{id}`
- `POST /api/admin/brokers`
- `PUT /api/admin/brokers/{id}`
- `POST /api/admin/brokers/{id}/activate`
- `POST /api/admin/brokers/{id}/deactivate`

9. Testing plan (separate: unit & integration)

9.1 Unit tests

Goal: Verify **business logic** in isolation, especially **policy calculations and state changes**.

Focus on:

1. **Policy Calculation Service**
 - Given a base premium and fee configurations, verify final premium is correctly computed.

- Scenarios:
 - No fees, no risk factors → final premium = base premium.
 - Multiple percentage adjustments → result uses correct formula.
 - Inactive or out-of-date configurations are ignored.
- 2. **Policy state transitions**
 - Draft → Active:
 - Valid input: status changes, activation date set.
 - Invalid input: start date in the past (if business rule forbids), missing mandatory fields.
 - Active → Cancelled:
 - Only allowed from Active.
 - Cancellation date required.

What to mock for unit tests:

- Data access abstractions (clients, policies, fees, risk factors, geography).
 - Logging interfaces.
-

9.2 Integration tests

Goal: Verify the Web API endpoints work correctly **end-to-end**: routing, model binding, validation, and database persistence.

Suggested integration tests:

1. **Create client + building + policy full flow**
 - Arrange:
 - Start with empty database (or known seed: countries, counties, cities, currencies, fees).
 - Steps:
 - Create a client via API.
 - Register a building for that client.
 - Create a draft policy for that client and building.
 - Activate the policy.
 - Assert:
 - All responses return expected HTTP status codes.
 - Policy is persisted with correct final premium and links to client, building, broker.
2. **Invalid policy creation**
 - Try creating a policy with:
 - Non-existent client or building.
 - Negative premium.
 - Assert:
 - API returns 400 Bad Request with validation messages.

- No policy is created in the database.
- 3. **Broker deactivation behaviour**
 - Create a broker and policy.
 - Deactivate broker via admin endpoint.
 - Attempt to create a new policy using the deactivated broker.
 - Assert:
 - API rejects the operation (e.g., 400 or 403).
 - Existing policies remain unaffected.

Integration test considerations:

- Use an **in-memory or test-specific database**.
 - Run setup before tests.
 - Clean database state between tests.
-

10. Acceptance criteria (checklist)

For the exercise to be considered complete:

1. **Core CRUD**
 - Brokers can create and manage clients, buildings, and policies via API.
 - Admins can manage currencies, fee configurations, risk factors, and brokers.
2. **Policy logic**
 - Policy creation applies fee and risk factor configurations to compute final premium.
 - Status transitions (Draft → Active → Cancelled) follow business rules.
3. **Quality**
 - Code is structured in clear layers (domain, application/services, infrastructure, Web).
 - Validation is implemented and tested.
 - Centralised error handling present.
4. **Tests**
 - Unit tests cover policy calculations and state transitions.
 - Integration tests cover main flows and reports.
 - Tests are easy to run from the solution.